

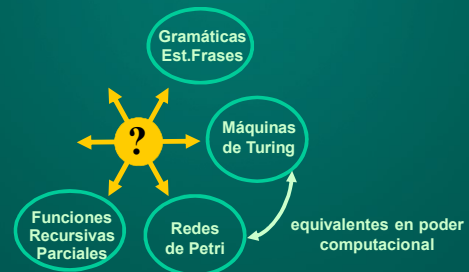
Teoría de la Computabilidad

Módulo 17: Teoremas de Equivalencia Funciones Recursivas Parciales Máquinas de Turing

Departamento de Cs. e Ing. de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

1

Equivalencia entre formalismos



2

Equivalencia entre formalismos



3

Toda FRP tiene una MT equivalente

Lema: Toda fc. recursiva parcial puede calcularse a través de una máquina de Turing.

→ Asumimos que una tupla de valores (x,y,z) se representa en la cinta a través de un *separator*. Ej:

$\#x\#y\#z\#$

→ Una MT $M=(S, \Sigma, \delta, s_0, F)$ calcula la función parcial $f: (\Sigma^*)^n \rightarrow \Sigma^n$ si para cada $(w_1, w_2, \dots, w_n) \in (\Sigma^*)^n$, al poner en marcha la MT con la cinta conteniendo $\#w_1\#w_2\#\dots\#w_n\#$ sucede que:

1. $f(w_1, w_2, \dots, w_n)$ no está definida. En ese caso M no termina su computación.
2. $f(w_1, w_2, \dots, w_n)=v$ está definida. Luego M se detiene con la respuesta $\#v\#$.

4

Funciones Cero, Succ e Identidad Gen.

→ Para simular $\text{cero}: \text{Nat}^n \rightarrow \{0\}$, $\text{cero}(x_1 \dots x_n)=0$, basta que M escriba $\#1\#$ y se detenga.

→ Para simular $\text{succ}: \text{Nat} \rightarrow \text{Nat}$, $\text{succ}(x)=x+1$, la MT M recibe x escrito en notación unaria (ej: $\#1111\#$ = 3) y debe agregar un 1 y detenerse.

→ Para simular $\Pi_i^n: \text{Nat}^n \rightarrow \text{Nat}$, $\Pi_i^n(x_1 \dots x_n) = x_i$ la MT M recibe n cadenas $\#x_1\#x_2\#\dots\#x_n\#$. La fc. de transición δ fuerza desplazamientos según el parám. i , ubicándose sobre $\#x_i\#$.

Se tacha todo a la derecha de $\#x_i\#$, y se mueve a izq. de x_i para tachar el resto. Resultado:
...##### x_i #####...

5

Fcs. Constructoras: Composición

Debemos probar que si $h: \text{Nat}^m \rightarrow \text{Nat}$ y $g_i: \text{Nat}^n \rightarrow \text{Nat}$ son fcs. Turing-computables, ent. $f(x_1 \dots x_n) = [h^\circ(g_1, \dots, g_m)](x_1 \dots x_n)$ también lo es.

Sea M una MT que computa h y las MT $M_1, M_2 \dots M_m$ que computan $g_1, \dots, g_m$

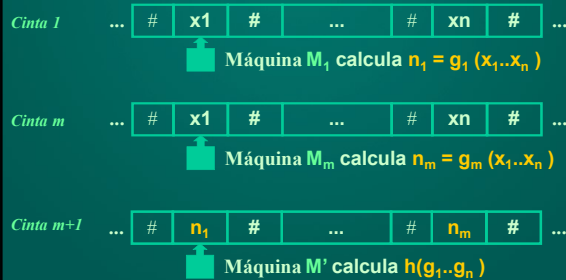
Podemos asumir que cada M_i toma como entrada una n -upla.

Puede construirse M' de $m+1$ cintas para calcular $h(g_1, \dots, g_m)$ como sigue: en la cinta i , $1 \leq i \leq m$, M' simula la acción de MT M_i . Si todas MT M_i paran, sus salidas se copian adecuadamente en la cinta $m+1$.

M' calcula luego $h(g_1, \dots, g_m)$ usando esa cinta.

6

Fcs. Constructoras: Composición



M' tiene m+1 cintas, y simula $M_1, M_2 \dots M_m$ en sus primeras m cintas, y en la cinta m+1 calcula $h(g_1..g_m)$

Fcs. Constructoras: Recursión Primitiva

Supongamos que $f: \text{Nat}^{m+1} \rightarrow \text{Nat}$ está definida por rec.primitiva. Esto es:

$$a) f(x_1..x_n, 0) = g(x_1..x_n)$$

$$b) f(x_1..x_n, m+1) = h(x_1..x_n, m, f(x_1..x_n, m))$$

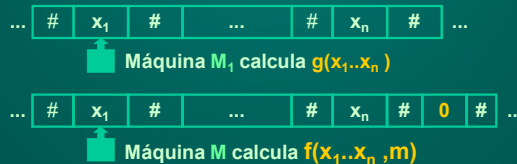
donde g es una fc.parcial calculada por M_1 y h es una fc.parcial calculada por M_2 . Veamos que f también puede calcularse con una MT M .

Caso a): si el último componente de la entrada es 0, lo borra, devuelve la cabeza a la celda del extremo izquierdo y simula M_1

Fcs. Constructoras: Recursión Primitiva

$$a) f(x_1..x_n, 0) = g(x_1..x_n)$$

$$b) f(x_1..x_n, m+1) = h(x_1..x_n, m, f(x_1..x_n, m))$$



La máquina M , en este caso donde $m=0$, borra las celdas que contienen m y luego simula lo que hace M_1 .



Fcs. Constructoras: Recursión Primitiva

$$a) f(x_1..x_n, 0) = g(x_1..x_n)$$

$$b) f(x_1..x_n, m+1) = h(x_1..x_n, m, f(x_1..x_n, m))$$

Nótese cómo calcular $f(x_1..x_n, y+1)$:

$$f(x_1..x_n, y+1) = h(x_1..x_n, y, f(x_1..x_n, y))$$

$$f(x_1..x_n, y) = h(x_1..x_n, y-1, f(x_1..x_n, y-1))$$

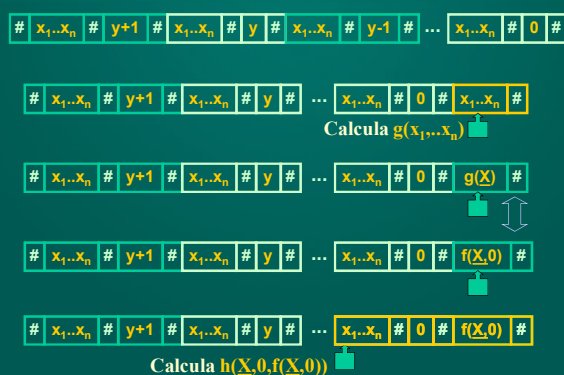
....

$$f(x_1..x_n, 2) = h(x_1..x_n, 1, f(x_1..x_n, 1))$$

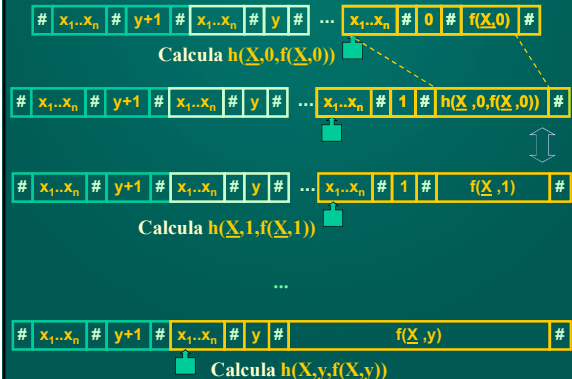
$$f(x_1..x_n, 1) = h(x_1..x_n, 0, f(x_1..x_n, 0))$$

$$f(x_1..x_n, 0) = g(x_1..x_n)$$

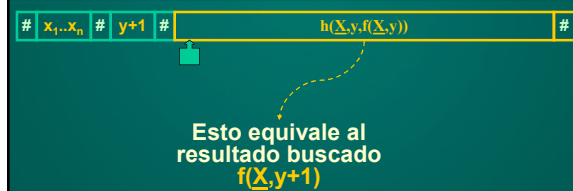
Fcs. Constructoras: Recursión Primitiva



Fcs. Constructoras: Recursión Primitiva



Fcs. Constructoras: Recursión Primitiva



13

Fcs. Constructoras: Minimización μ_y

- Finalmente veamos que sucede con la **minimización no acotada** μ_y . Debemos mostrar que la misma también puede hallarse con una MT.

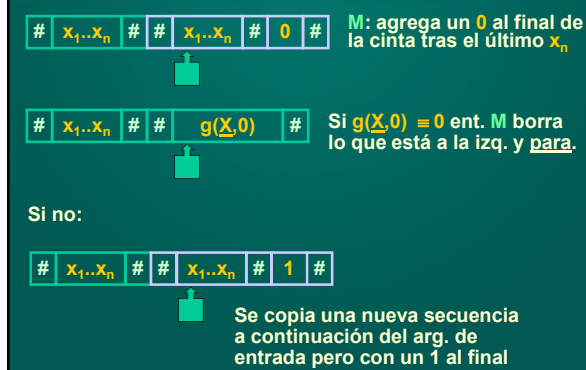
- Sea $f(\underline{X}) = \mu_y [g(\underline{X}, y)=0]$

donde g es una fc. calculada por una MT M_g . Construiremos una MT M usando el sgte. algoritmo:

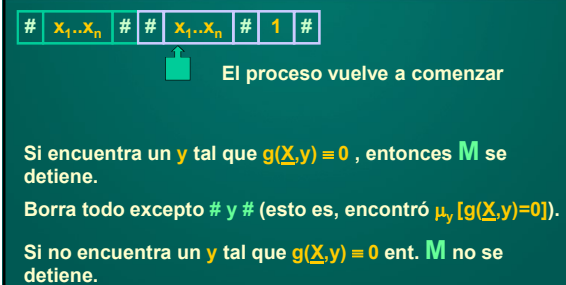
$x_1 \dots x_n$ # ($x_1 \dots x_n$ es en realidad $\#x_1\#x_2\#\dots\#x_n$)

Máq M : duplica los args. de entrada dejando dos símbolos # como separación

14

Fcs. Constructoras: Minimización μ_y 

15

Fcs. Constructoras: Minimización μ_y 

16

Toda MT es en realidad una FRP

Mostramos que toda función recursiva parcial es computable por una máquina de Turing.

Una máquina de Turing cualquiera ¿puede ser expresada por una función recursiva parcial?

¡Si!

Lema: Todo proceso computacional desarrollado por una Máquina de Turing es en realidad el cálculo de una función recursiva parcial.

Consideraremos una MT $M=(S, \Sigma, \delta, s_0, \{s_i\})$ y codificaremos la cinta de M usando Nat . El cómputo de M puede entenderse como una fc. $f:Nat \rightarrow Nat$. Debemos mostrar que f es FRP.

17

Toda MT es en realidad una FRP

Lema: Todo proceso computacional desarrollado por una Máquina de Turing es en realidad el cálculo de una función recursiva parcial.

Demostración:

Sea MT $M=(S, \Sigma, \delta, s_0, \{s_i\})$ y sea $f:Nat \rightarrow Nat$ una fc. calculada por M . $f(n)$ está definida para $n \in Nat$ sssi M se detiene con $f(n)$ en la cinta tras haber comenzado con n como argumento de entrada.

Sin pérdida de generalidad, podemos asumir que la MT es determinista y está completamente especificada.

18

Toda MT es en realidad una FRP

Intuición de la Demostración:

Imaginemos un programa en Pascal que calcula $f: \text{Nat} \rightarrow \text{Nat}$, implementándola. Entonces f debe poder calcularse usando funciones auxiliares que *imiten* el comportamiento de una máquina de Turing.

Todo este comportamiento deberá modelarse utilizando FRPs.

Debemos definir en forma abstracta una Máquina de Turing usando distintas FRPs.

19

Función Estado: $\text{Nat}^2 \rightarrow \text{Nat}$

$$\text{estado}(p,x) = \begin{cases} q \ (0 \leq q < k) & \text{si } M \text{ cambia al estado } q \\ & \text{cuando está en estado } p \text{ y símbolo actual } x \\ k & \text{si } (p,x) \text{ no es un par válido} \end{cases}$$

Codificación de nro. de estado:

Estado 0: estado de detención de M

Estado 1,2,...k-1: estados comunes

$$\begin{array}{lcl} \dots \# & a & \# \dots \\ & \uparrow & \\ & s_3 & \end{array} \quad \begin{array}{l} s_3 \Rightarrow 3 \\ s_4 \Rightarrow 4 \\ a \Rightarrow 21 \end{array}$$

$$\delta(s_3, a, s_4, L) \quad \text{estado}(3, 21) = 4$$

20

Fcs. Simb: $\text{Nat}^2 \rightarrow \text{Nat}$ y Mov: $\text{Nat}^2 \rightarrow \text{Nat}$

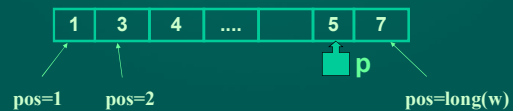
$$\text{Simb}(p,x) = \begin{cases} y & \text{si } M \text{ escribe } y \text{ cuando se halla} \\ & \text{en estado } p, \text{ con símbolo actual } x \\ x & \text{en los demás casos} \end{cases}$$

$$\text{Mov}(p,x) = \begin{cases} 2 & \text{si } M \text{ mueve su cabeza a} \\ & \text{derecha cuando está en} \\ & \text{estado } p, \text{ símbolo actual } x \\ 1 & \text{si } M \text{ mueve su cabeza a} \\ & \text{izquierda cuando está en} \\ & \text{estado } p, \text{ símbolo actual } x \\ 0 & \text{si } M \text{ escribe sin} \\ & \text{desplazamiento.} \end{cases}$$

21

Representando Configuraciones de M

$$(p,w,n) = \begin{cases} p & = \text{nro. natural que representa} \\ & \text{el estado actual de } M. \\ w & = \text{contenido de la cinta} \\ n & = \text{posición en la cinta} \\ & \text{contando de izq. a der.} \end{cases}$$

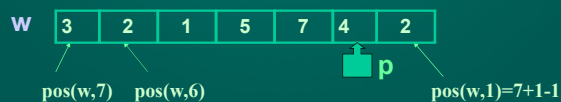


22

Función pos: $\Sigma^* \times \text{Nat} \rightarrow \text{Nat}$

Asumiremos en lo sucesivo que la posición es dada de derecha a izquierda. Para ello definiremos una fc. **Pos** para indicar posición como sigue:

$$\text{pos}(w,n) =_{\text{def}} \text{long}(w) + 1 - n$$



Cinta w contiene el natural: 2475123

23

Función SimbAct: $\text{Nat} \times \text{Nat} \rightarrow \text{Nat}$

Sea b =base del sistema numérico usado para codificar el contenido de la cinta. Esta base será elegida acorde al alfabeto Σ utilizado.

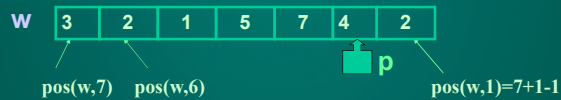
La función **SimbAct** calcula el símbolo actual a partir de (p,w,n) , y se define como sigue:

SimbAct(w,n)=

$$\text{div}(w, b^{\text{Pos}(w,n)+1}) \div \text{mult}(b, \text{div}(w, b^{\text{Pos}(w,n)}))$$

Nota: se asume que las operaciones aritméticas se hacen en base b .

24

Ejemplo: Función SimbAct

Cinta w contiene el natural: 2475123

$$\text{SimbAct}(w,n) = \text{div}(w, b^{\text{Pos}(w,n)+1}) \div \text{mult}(b, \text{div}(w, b^{\text{Pos}(w,n)}))$$

Ejemplo: supongamos $b=10$, $\text{long}(w)=7$, y querer obtener $\text{SimbAct}(w,2)$ según la cinta de arriba. Ent: $\text{Pos}(w,2) \stackrel{\text{def}}{=} \text{long}(w)+1+2$ luego $\text{Pos}(w,2)=7+1+2=6$

25

Ejemplo: Fc. SimbAct

W

3	2	1	5	7	4	2
---	---	---	---	---	---	---

Cinta w contiene el natural: 2475123

$$\text{SimbAct}(w,n) = \text{div}(w, b^{\text{Pos}(w,n)+1}) \div \text{mult}(b, \text{div}(w, b^{\text{Pos}(w,n)}))$$

Ejemplo: supongamos $b=10$, $\text{long}(w)=7$, $n=2$. Queremos obtener $\text{SimbAct}(w,2)$ según la cinta de arriba. Entonces: $\text{Pos}(w,2)=6$

$$\begin{aligned} \text{SimbAct}(w,2) &= \text{div}(w, 10^{6+1}) \div \text{mult}(10, \text{div}(w, 10^6)) = \\ &= \text{div}(2475123, 10^5) \div \text{mult}(10, \text{div}(2475123, 10^6)) = \\ &= 24 \div \text{mult}(10, 2) = 24 \div 20 = 4 \end{aligned}$$

26

Función CabezaSig: $\text{Nat}^3 \rightarrow \text{Nat}$

La función **CabezaSig** devuelve un entero positivo (=posición siguiente de la cabeza en la cinta). Si **CabezaSig** devuelve 0, indica terminación anómala.

$$\begin{aligned} \text{CabezaSig}(p,w,n) &= n \div \\ &f_{\text{Igual}}(\text{mov}(p, \text{simbact}(w,n)), 1) + \\ &f_{\text{Igual}}(\text{mov}(p, \text{simbact}(w,n)), 2) \end{aligned}$$

Intuición: Si $\text{mov}(p, \text{simbact}(w,n))=1$, ent. la cabeza se mueve a izquierda; si $\text{mov}(p, \text{simbact}(w,n))=2$, entonces se mueve a derecha.

27

Función EstadoSig: $\text{Nat}^3 \rightarrow \text{Nat}$

La función **EstadoSig** devuelve un nro. de estado válido (estado al cual pasaría **M** a partir de la configuración (p,w,n)).

$$\begin{aligned} \text{EstadoSig}(p,w,n) &= \\ &\text{estado}(p, \text{simbact}(w,n)) + \\ &\text{mult}(k, \text{sg}'(\text{cabezasig}(p,w,n))) \end{aligned}$$

Nota: $\text{sg}'(n) = 1$ si $n=0$, $\text{sg}'(n)=0$ si $n>0$.

Nota: obsérvese que si $\text{cabezasig}(p,w,n)=0$ entonces **EstadoSig** (p,w,n) devuelve estado $j>k$ (=estado no-válido)

28

Función CintaSig: $\text{Nat}^3 \rightarrow \text{Nat}$

La fc. **CintaSig** devuelve el entero que representa la nueva cinta de **M** después de ejecutar una transición a partir de (p,w,n) .

$$\begin{aligned} \text{CintaSig}(p,w,n) &= \\ &w \div \text{mult}(b^{\text{Pos}(w,n)+1}, \text{simbact}(w,n)) + \\ &\text{mult}(b^{\text{Pos}(w,n)+1}, \text{simb}(p, \text{simbact}(w,n))) \end{aligned}$$

29

Ejemplo: Función CintaSig

W

3	2	1	5	7	4	2
---	---	---	---	---	---	---

Cinta w contiene el natural: 2475123

Ej: supongamos $b=10$, $\text{long}(w)=7$, $n=2$. Ent. $\text{SimbAct}(w,2)=2$. Supongamos que estado actual $p=7$, y $\text{Simb}(7,4)=8$ (escribe 8 sobre el 4).

$$\begin{aligned} \text{CintaSig}(p,w,n) &= \\ 2475123 \div \text{mult}(10^5, \text{simbact}(w,n)) + \\ \text{mult}(10^5, \text{simb}(p, \text{simbact}(w,n))) &= \\ 2475123 \div 400000 + 800000 &= 2875123 \end{aligned}$$

30

Función Paso: $\text{Nat}^3 \rightarrow \text{Nat}^3$

La función **Paso** relaciona una terna asociada a una configuración con otra terna (=configuración siguiente).

$$\text{Paso}(p, w, n) = (\text{EstadoSig}(p, w, n), \\ \text{CintaSig}(p, w, n), \\ \text{CabezaSig}(p, w, n))$$

31

Función Ejec: $\text{Nat}^4 \rightarrow \text{Nat}^3$

La función **Ejec: $\text{Nat}^4 \rightarrow \text{Nat}^3$** representa la situación a la que arriba **M** después de ejecutar **t+1** transiciones desde la configuración de arranque.

$$\text{Ejec}(p, w, n, 0) = (p, w, n)$$

$$\text{Ejec}(p, w, n, t+1) = (\text{paso} \circ \Pi_5^5)(p, w, n, t, \text{ejec}(p, w, n, t))$$

Estado actual
0 = estado
detención

Posición en la cinta

Cadena en la cinta

Observación: Nótese que el dominio de **Ejec** es una 4-upla, pero su imagen es una 3-upla.

32

Función Cant_Pasos

La función **Cant_Pasos: $\text{Nat} \rightarrow \text{Nat}$** representa la cantidad de pasos necesarios para arribar al estado de detención. Informalmente:

$$\text{Cant_Pasos}(w) = \mu_t[\Pi_1^3(\text{ejec}(1, w, 1, t)) = 0]$$

(el mínimo **t** que hace que el 1er. elemento devuelto por **ejec** sea 0). Recordemos que **ejec: $\text{Nat}^4 \rightarrow \text{Nat}^3$** devuelve una 3-upla (p, w, n) , y que $p=0$ significa “estado de detención”.

Más formalmente:

$$\text{Cant_Pasos}(w) =$$

$$\mu_t[\text{f}_{\text{igual}}[\Pi_1^3(\text{ejec}(1, \Pi_1^2, 1, \Pi_2^2), \text{cero}())]](w, t)$$

33

Función f_M

Finalmente podemos expresar la MT **M** mediante la fc. recursiva parcial **f: $\text{Nat} \rightarrow \text{Nat}$** . Informalmente:

$$f(w) = \Pi_2^3(\text{ejec}(1, w, 1, \text{cant_Pasos}(w)))$$

Interpretación: como **ejec: $\text{Nat}^4 \rightarrow \text{Nat}^3$** devuelve una 3-upla (p, w, n) , se está devolviendo la cadena resultante **w** (=resultado de la ejecución de la MT **T**) después de haberse ejecutado cierta cantidad de pasos y haber arribado a un estado aceptador.

Más formalmente:

$$f(w) = \Pi_2^3[\text{ejec}(1, \Pi_1^1, \text{cant_Pasos}(\Pi_1^1))](w)$$

34

Teorema de Kleene: Una fc. es recursiva parcial definida sobre **Nat** sssi es una función Turing computable sobre **Nat**.



35

FRPs y Lenguajes de Programación

Mostraremos finalmente que los distintos constructores teóricos provistos por FRPs pueden representarse usando un Lenguaje de Programación (ej: Pascal).

Debemos mostrar trozos de código Pascal que simulen el comportamiento de :

- las distintas funciones iniciales
- aquellas FRPs obtenibles mediante constructores (rec.prim, composición, minimización no acotada).

36

FRPs y Lenguajes de Programación

Funciones Iniciales:

 $z := 0 \rightarrow \text{fc. cero}$ $z := z+1 \rightarrow \text{fc. sucesor}$ $z := x[i] \rightarrow \text{fc. } \Pi_1^n$

Composición de funciones:

 $z_1 := g_1(x_1 \dots x_n)$

....

 $z_m := g_m(x_1 \dots x_n)$ $f := h(z_1 \dots z_m)$

37

FRPs y Lenguajes de Programación

Recursión Primitiva: consideraremos una forma simplificada de la misma. $f(0) = k$ $f(x+1) = h(x, f(x))$ Nótese que z va desde 0 hasta x . x es una variable usada como argumento a decrementar para implementar la recursión. y toma valores $h(0, f(0)), h(1, f(1)), \dots, h(x, f(x)) = f(x+1)$.

```

read(x)
z:=0
y:=k
While x > 0 do
begin
  y := h(z,y)
  z:=z+1
  x:=x-1
end;
f:=y

```

38

FRPs y Lenguajes de Programación

Minimizac. No Acotada: basta el trozo de código que se muestra:

La **minimización** está representada en los LPs por la capacidad de realizar ciclos o bucles con una condición de terminación (=ciclos **WHILE** o **REPEAT..UNTIL**).

Recursión primitiva: involucra un bucle, pero el número de pasos está predeterminado.

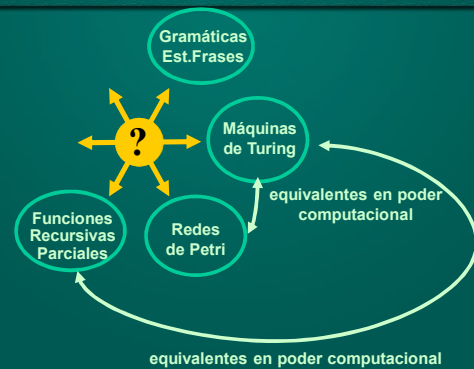
```

read(x)
y:=0
While g(x,y) ≠ 0 do
  y :=y+1
f:=y

```

39

Síntesis



40